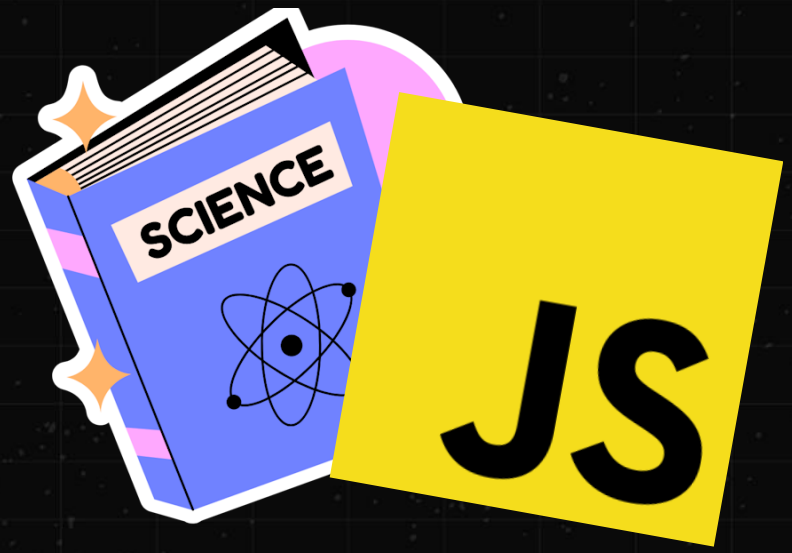
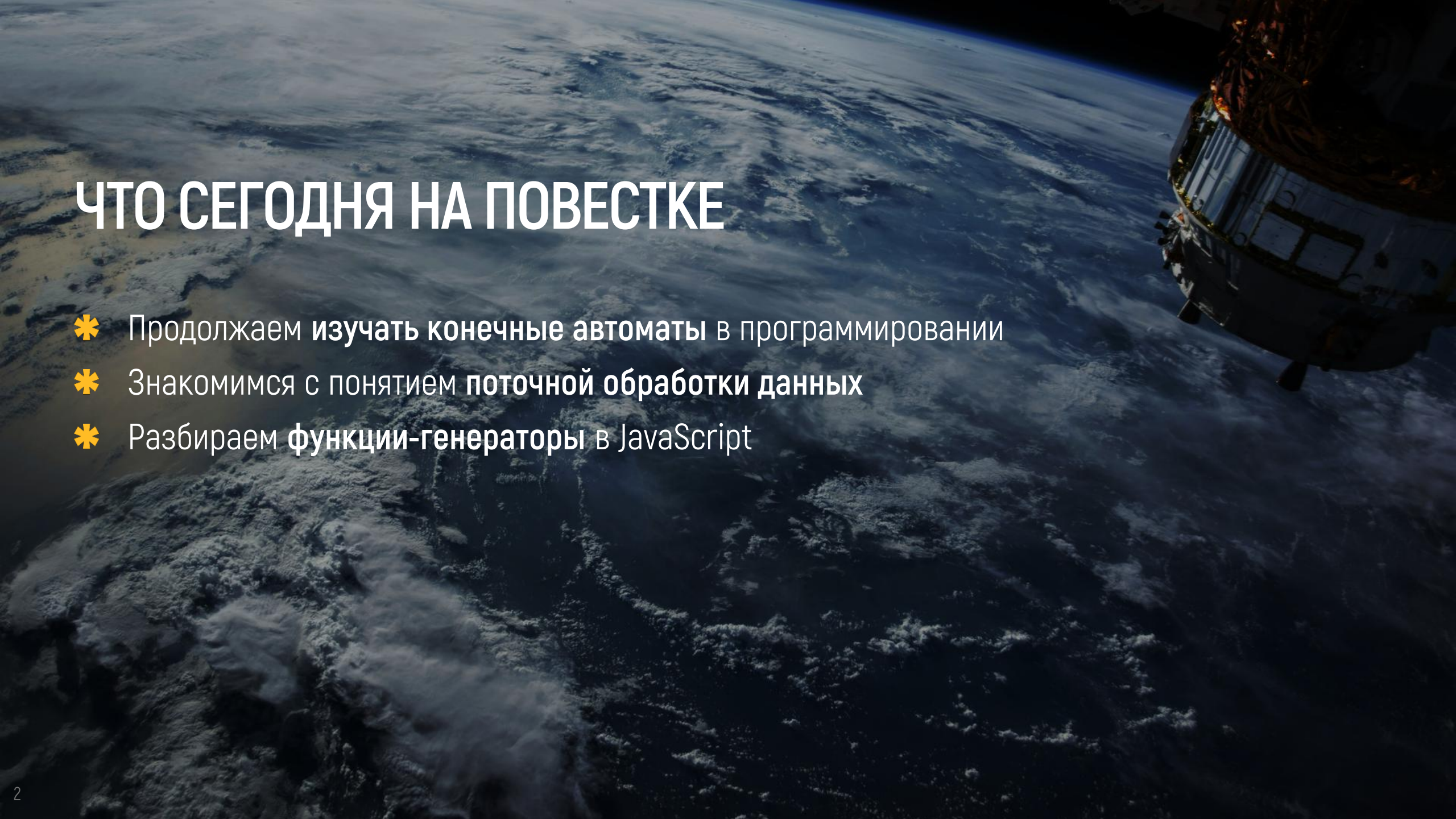
KOBEZZA.LAB

ЛЕКЦИЯ #25

CS BO FRONTEND





ЧТО СЕГОДНЯ НА ПОВЕСТКЕ

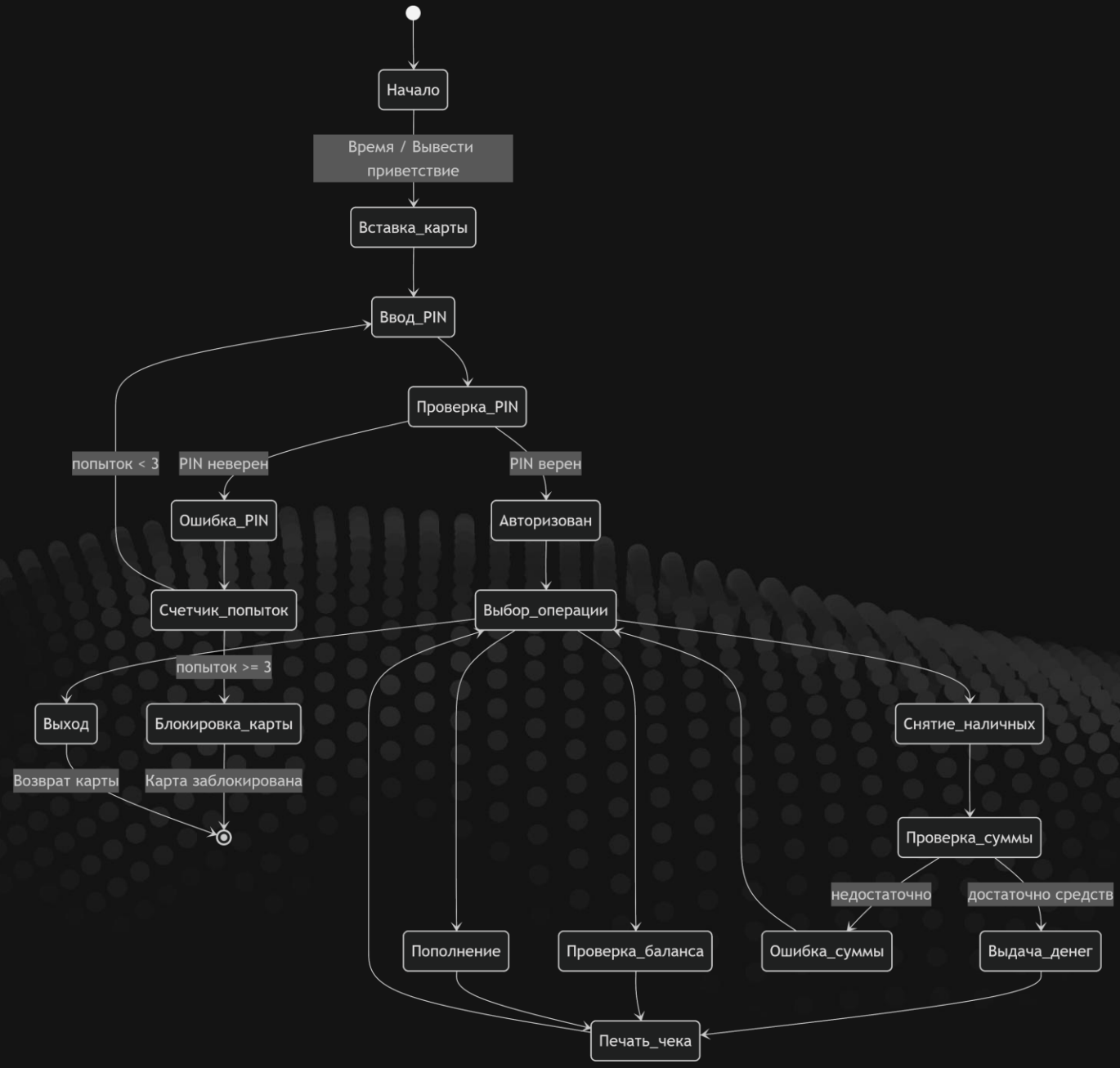
- Продолжаем **изучать конечные автоматы** в программировании
- Знакомимся с понятием **поточной обработки данных**
- Разбираем **функции-генераторы** в JavaScript

АВТОМАТ

- Это устройство, имеющее **вход, выход и внутреннее состояние**
- Устройство может за один шаг **перейти из одного состояния в другое**
- Переход осуществляется на основании входных данных и текущего состояния
- Состояние, **после которого переходы становятся невозможными,** называется терминальным (финальным)

КОНЕЧНЫЙ АВТОМАТ

- Это автомат, **количество состояний которого конечно** и строго определено
- **Если все переходы автомата строго определены** в зависимости от текущего состояния и входного значения, то такой автомат называется детерминированным


Начало
Время / Вывести приветствие
Вставка_карты
Ввод_PIN
Проверка_PIN
попыток < 3
PIN неверен
PIN верен
Ошибка_PIN
Авторизован
Счетчик_попыток
Выбор_операции
попыток >= 3
Выход
Блокировка_карты
Возврат карты
Карта заблокирована
Снятие_наличных
Проверка_суммы
недостаточно
достаточно средств
Пополнение
Проверка_баланса
Ошибка_суммы
Выдача_денег
Печать_чека

КА ДЛЯ ПРОГРАММИСТА

- Объект или функция, **имеющие состояние**
- Работа функций или методов **зависит от входных данных** и состояния
- **Состояние может сохраняться** между вызовами функций, но это не обязательно

```
function createLightSwitch() {
    let state = "off";

    return function toggle() {
        state = (state === "off") ? "on" : "off";
        return state === "on" ? " 💡 Включено" : " 💡 Выключено";
    };
}

const switchMachine = createLightSwitch();
console.log(switchMachine()); // 💡 Включено
console.log(switchMachine()); // 💡 Выключено
console.log(switchMachine()); // 💡 Включено
```

КОГДА НУЖНО ЯВНО ИСПОЛЬЗОВАТЬ КА?



ЕСТЬ ПРОСТОЙ ПРИЗНАК

- Когда у вас есть последовательность логики, **которая зависит от предыдущего состояния**
- Функция регулировки яркости освещения
- Логика работы NPC в компьютерной игре
- Парсер для некоторого языка
- Последовательность асинхронных обработчиков (например, DnD)
- Транзакция между объектами

```
function createBrightness() {
    let state = 0; // 0-100%

    return function next(step = 10) {
        let newState = Math.min(100, Math.max(0, state + step));
        state = newState;
        return `💡 Яркость: ${state}%`;
    };
}

const brightness = createBrightness();
console.log(brightness(20));  // 💡 Яркость: 20%
console.log(brightness(30));  // 💡 Яркость: 50%
console.log(brightness(-10)); // 💡 Яркость: 40%
```

СОСТОЯНИЕ АВТОМАТА ХРАНИТСЯ В ПАМЯТИ

- Это могут быть **обычные переменные или свойства**, которые меняют значения
- А **может быть и стек**, который хранит историю состояний
- Такие автоматы называют автоматами с магазинной памятью (стековые автоматы)
- Можно также хранить память в виде ассоциативного массива
- Всё зависит от задачи

```
function createBracketChecker() {
    let stack = [];

    return function next(char) {
        if (char === "(" || char === "[" || char === "{") {
            stack.push(char);
            return `📥 Открывающая: ${char}, стек: [${stack.join(", ")}]`;
        }

        if (char === ")" || char === "]" || char === "}") {
            const last = stack.pop();
            const pairs = { "(": ")", "[": "]", "{": "}" };

            if (pairs[last] !== char) {
                return `Ошибка: несовпадение "${char}"`;
            }

            return `Закрывающая: ${char}, стек: [${stack.join(", ")}]`;
        }

        return `Игнорируем: "${char}"`;
    };
}

const checker = createBracketChecker();
console.log(checker("("));  // Открывающая: (, стек: [(]
console.log(checker("{"));  // Открывающая: {, стек: [(, {]
console.log(checker("}"));  // Закрывающая: }, стек: [(]
console.log(checker(")"));  // Закрывающая: ), стек: []
```


second monitor
$0.00 (0%)
$0.00
224 DAYS TO 20
$160.00
СЛОЖНА, СЛОЖНАААА!

ВЫДЫХАЕМ

- Мы уже сталкивались с автоматами при изучении регулярных выражений
- По сути, любое регулярное выражение компилируется в некоторый автомат
- **Итераторы – это ещё один пример автомата**, где результат вызова `next()` зависит от текущего состояния итератора

```
function createRangeIterator(start, end) {
    let current = start; // Состояние
    return {
        next() { // Переход
            if (current > end) {
                return { value: undefined, done: true };
            }
            return { value: current++, done: false };
        }
    };
}

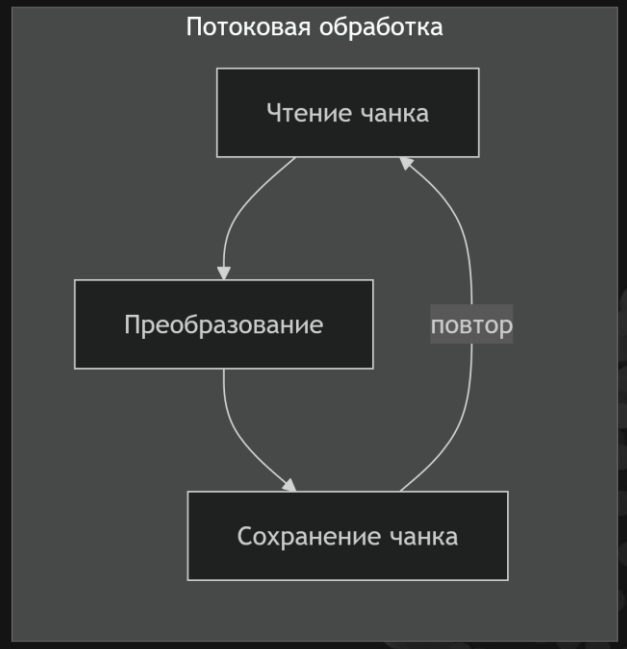
const iter = createRangeIterator(1, 3);
console.log(iter.next()); // { value: 1, done: false } → состояние: 2
console.log(iter.next()); // { value: 2, done: false } → состояние: 3
console.log(iter.next()); // { value: 3, done: false } → состояние: 4
console.log(iter.next()); // { done: true } → терминальное состояние
```

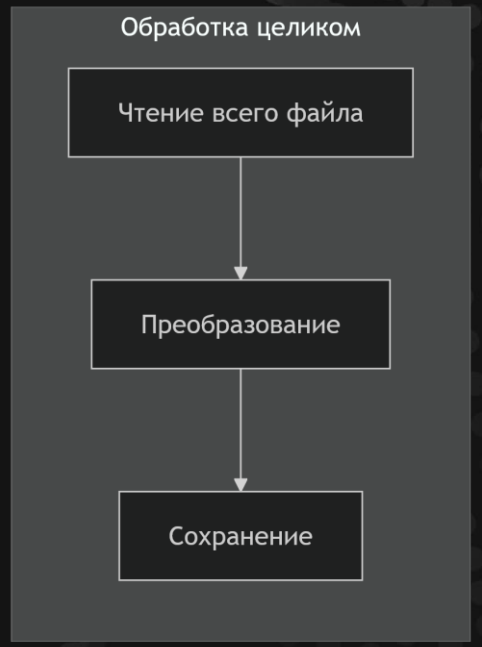
РАЗВИВАЯ ИДЕЮ ИТЕРАТОРОВ

- Итераторы позволяют описывать **последовательности данных на основе других данных**
- При каждом вызове `next()` мы получаем новую порцию данных
- Ленивость итераторов **даёт потребителю возможность решать,** когда получить следующую порцию данных
- К любому итератору легко добавить адаптер – он **создаст новый итератор на основе исходного**

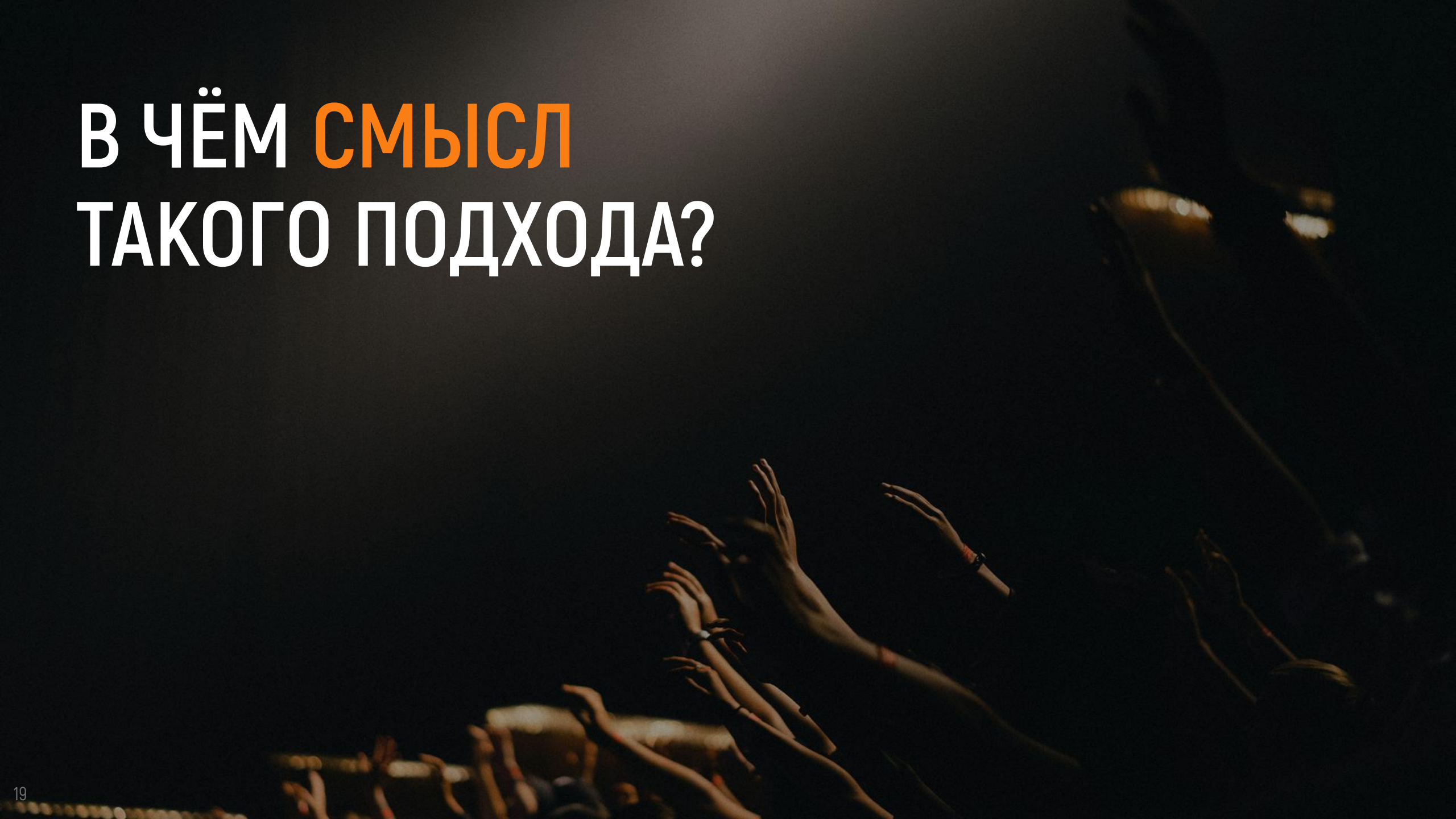
ПОТОК ДАННЫХ

- Цепочку итераторов можно **представить как некоторый поток данных**
- Главное отличие в том, что при обычном подходе данные получают сразу целиком, а **в потоке они появляются и обрабатываются частями**
- Причём **целиком данные зачастую и не нужны** вовсе
- Например, чтобы преобразовать текстовый файл в верхний регистр и сохранить его на диск, совсем не нужно предварительно читать весь файл в память
- Можно **постепенно читать данные,** преобразовывать их и сразу сохранять на диск полученный фрагмент


Потоковая обработка
Чтение чанка
Преобразование
повтор
Сохранение чанка


Обработка целиком
Чтение всего файла
Преобразование
Сохранение

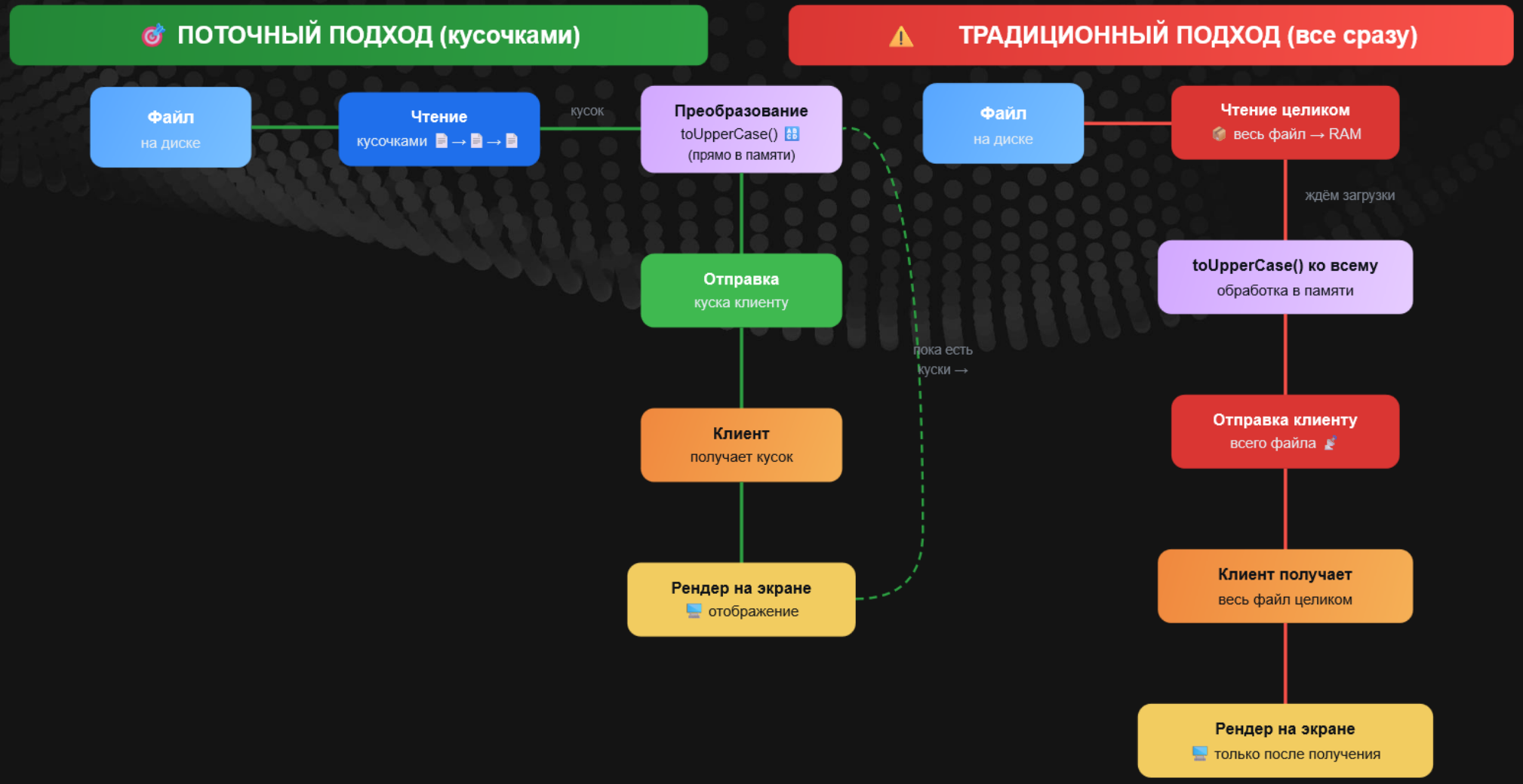
В ЧЁМ СМЫСЛ ТАКОГО ПОДХОДА?



КОНТРОЛЬ РЕСУРСОВ

- При выполнении операции «целиком» может потребоваться **значительный объём памяти и процессорного времени**
- Более того, многие источники данных не способны предоставить их моментально — данные поступают постепенно, например, при чтении с диска или передаче по сети
- Можно **не дожидаться получения всех фрагментов,** а начать обработку сразу по мере поступления данных

Сравнение подходов к обработке данных



✅ Низкое потребление памяти
✅ Ранний старт рендеринга
✅ Клиент видит результат сразу

❌ Высокое потребление памяти
❌ Задержка до полной загрузки
❌ Клиент ждёт всё или ничего

БЕСКОНЕЧНЫЕ ИСТОЧНИКИ

- Существуют источники данных, которые **генерируют данные бесконечно**
- **Работать с такими данными можно только в виде потока,**
  так как в принципе отсутствует понятие «данные закончились»
- Как и нет бесконечного количества памяти и ресурсов

ЗНАЧИТ ИТЕРАТОРЫ НУЖНЫ ДЛЯ ПОТОЧНОЙ РАБОТЫ С ДАННЫМИ?



НЕ СОВСЕМ ТАК

- Работать в поточной форме с данными можно, используя разные API
- Концептуально можно разделить реализации: **событийную и императивную**
- В событийном подходе **данные приходят автоматически по некоторому событию**
- В императивном подходе **для чтения данных используются явные вызовы** API
- Работа с потоком через итераторы как раз использует императивный подход

```
import { createReadStream } from "node:fs";

// Событийный подход

const stream1 = createReadStream("file.txt");

stream1.on("data", (chunk) => {
  console.log("Получен чанк (событие):", chunk.toString());
});

stream1.on("end", () => {
  console.log("Чтение завершено (событие)");
});

// Императивный подход (упрощенный)

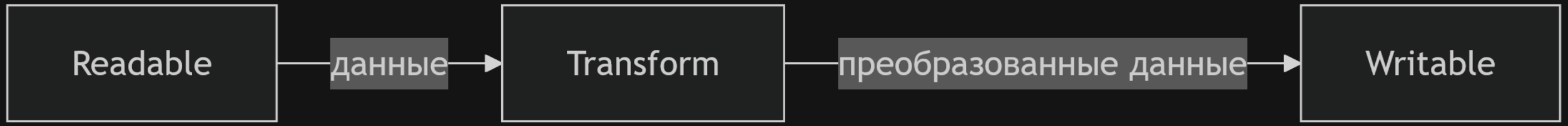
const stream2 = createReadStream("file.txt");

let chunk;
while ((chunk = stream2.read()) !== null) {
  console.log("📖 Прочитано (read):", chunk.toString());
}
```

РЕФЛЕКСИРУЕМ

- Неважно, какое API вы используете для создания потока данных — главное, что **работа с данными выполняется частями**
- Причём в потоке можно **не только читать, но и писать данные**
- В реальности потоки **разделяют на потоки чтения, записи и полнодуплексные**
- Также отдельно выделяют потоки трансформации, которые **могут преобразовывать данные после чтения и перед записью**

ВИЗУАЛИЗИРУЕМ



```
import { Transform } from "node:stream";

import { createReadStream, createWriteStream } from "fs";

const upperCaseTransform = new Transform({
    transform(chunk, encoding, callback) {
        const result = chunk.toString().toUpperCase();
        callback(null, result);
    }
});

createReadStream("input.txt")
    .pipe(upperCaseTransform)
    .pipe(createWriteStream("output.txt"))
    .on("finish", () => {
        console.log(" ✅ Преобразование завершено");
    });
```

НО ЗДЕСЬ ЖЕ НИГДЕ НЕТ ИТЕРАТОРОВ?



ДА, НО У ПОДХОДА НА ИТЕРАТОРАХ ЕСТЬ ПЛЮСЫ

- Действительно, **в Node.js и браузере есть отдельные API** для работы с потоками
- Они удобны, но **несовместимы друг с другом**
- С другой стороны, **у нас есть итераторы, которые являются частью языка,** а не среды выполнения
- И **работают везде предсказуемо одинаково**

ДАВАЙТЕ РАЗБИРАТЬСЯ

- Поток чтения – любой вызов next() по сути является аналогом read()
- К тому же мы можем использовать цикл for of, а не while
- **Если данные приходят асинхронно**, итератор может возвращать Promise
- **Адаптеры итераторов** позволяют легко выполнять трансформацию данных

```
import { openSync, readSync, closeSync } from "node:fs";

function createFileIterator(filePath, chunkSize = 64) {
    let position = 0;
    let isDone = false;

    const fd = openSync(filePath, "r");
    return Iterator.from({
        next() {
            while (true) {
                if (isDone) { return { value: undefined , done: true }; }

                const buffer = Buffer.alloc(chunkSize);
                const bytesRead = readSync(fd, buffer, 0, chunkSize, position);

                if (bytesRead === 0) {
                    closeSync(fd);
                    isDone = true;
                } else {
                    position += bytesRead;
                    return { value: buffer.subarray(0, bytesRead) , done: false };
                }
            }
        }
    });
}

const iter = createFileIterator("file.txt", 10)
    .map(String)
    .filter((text) => /[A-Z]/.test(text));

for (const chunk of iter) {
    console.log("Чанк: ", chunk.toString());
}
```


second monitor
$0.00 (0%)
$0.00
224 DAYS TO 20
$160.00
СЛОЖНА, СЛОЖНАААА!

ЧТО ЗДЕСЬ ВАЖНО ПОНЯТЬ

- Итераторы **естественным образом создают поток данных** по своему определению
- Более того, это один из самых простых способов представить поток
- Но синтаксически всё это выглядит сложновато
- К счастью, **в JavaScript есть специальный синтаксис** для простого создания таких конструкций

```
import { openSync, readSync, closeSync } from "node:fs";

function* createFileIterator(filePath, chunkSize = 64) {
    let position = 0;
    const fd = openSync(filePath, "r");

    while (true) {
        const buffer = Buffer.alloc(chunkSize);
        const bytesRead = readSync(fd, buffer, 0, chunkSize, position);

        if (bytesRead === 0) {
            closeSync(fd);
            return;
        }

        position += bytesRead;
        yield buffer.subarray(0, bytesRead);
    }
}

const iter = createFileIterator("file.txt", 10)
    .map(String)
    .filter((text) => /[A-Z]/.test(text));

for (const chunk of iter) {
    console.log("Чанк: ", chunk.toString());
}
```

ВСТРЕЧАЙТЕ, ГЕНЕРАТОРЫ

- Это **особый вид функций в JavaScript** и создаётся с ключевым словом function*
- Поддерживается и синтаксис для методов, например, {*example() {}}
- Их особенность в том, что они позволяют **вернуть результат более одного раза**
- Делается это с помощью операторов yield и yield*
- **При этом return, как и раньше,** полностью завершает выполнение функции

ВНЕШНИЙ ИНТЕРФЕЙС ГЕНЕРАТОРА

- **Результатом вызова функции-генератора** является итератор
- Каждый вызов yield будет обработан как **очередное значение итератора**
- **return также обрабатывается в формате итератора,** но со значением done: true
- Если генератор завершился, **перезапустить итератор уже не получится**

```
function* example() {
    yield 1;
    yield 2;
    return 3;   // Завершает генератор
    yield 4;    // Никогда не выполнится
}

const gen = example();
console.log(gen.next()); // { value: 1, done: false }
console.log(gen.next()); // { value: 2, done: false }
console.log(gen.next()); // { value: 3, done: true }
console.log(gen.next()); // { value: undefined, done: true }
```

СОСТОЯНИЕ МЕЖДУ yield

- Состояние не сбрасывается – все переменные остаются такими,
  как будто функция не завершилась, а просто поставилась на паузу
- Если вызвать yield внутри цикла, **то цикл «замрёт»** до следующего вызова next
- Но вызвать yield можно **только внутри непосредственной функции-генератора**

```
function* counter() {
    let count = 0; // состояние
    while (true) {
        yield count++;
    }
}

const gen = counter();
console.log(gen.next().value); // 0 (count = 0)
console.log(gen.next().value); // 1 (count = 1)
console.log(gen.next().value); // 2 (count = 2)
```

ПЕРЕДАЧА ДАННЫХ В next

- Итератор, созданный из функции-генератора,
  по умолчанию **может принимать данные** при вызове next(value)
- Эти данные можно получить **в том месте, где был вызван yield**
- Их можно **сохранить в переменную или передать аргументом** в функцию
- Это позволяет использовать такие **итераторы как потоки записи,** а не только чтения

```
function* logger() {
    while (true) {
        const message = yield; // Получение данных из next(message)

        if (message === "stop") {
            return "Логгер остановлен";
        }

        console.log(`Лог: ${message}`);
    }
}

const log = logger();
log.next(); // Остановились на const message = yield;

log.next("Hello World");         //  Лог: Hello World
log.next("Error occurred");      // Лог: Error occurred
console.log(log.next("stop")); // { done: true, value: 'Логгер остановлен' }
console.log(log.next());         // { done: true, value: undefined }
```


second monitor
$0.00 (0%)
$0.00
224 DAYS TO 20
$160.00
СЛОЖНА, СЛОЖНАААА!

ЧТО УЖЕ ПОНЯТНО

- Генераторы – это **фабрика для создания итераторов**
- Используя yield, мы по сути делаем возврат из итератора с done: false
- А возможность передать значение снаружи позволяет **переключать наш автомат** в новые состояния и использовать это для решения большего числа задач
- Конечно, мы **могли бы написать такой итератор и без генератора,** но не так удобно
- Так что можно сказать, что генераторы – это **«синтаксический сахар» для описания автоматов**

НЕ ТОЛЬКО next

- Итератор, созданный через генератор, также имеет свои методы return и throw
- При этом вызов return по умолчанию переводит итератор в done: true
- Это не всегда удобно, так как любой break в цикле по итератору может навсегда прервать обход – поэтому **есть смысл переопределить это поведение**
- Такой итератор также **наследует все остальные методы итераторов,** такие как filter или map

```
function* random(from, to) {
  while (true) {
    yield Math.floor(Math.random() * (to - from) + from);
  }
}

const iter = random(10, 100);

for (const value of iter) {
  console.log(value); // 82
  break;                // Вызов return
}

// Итераций не будет из-за return в прошлом цикле
for (const value of iter) {
  console.log(value);
  break;
}

const iter2 = Object.assign(random(10, 100), { return: (value) => ({ value, done: true }) });

for (const value of iter2) {
  console.log(value); // 18
  break;
}

for (const value of iter2) {
  console.log(value); // 86
  break;
}
```

ИСКЛЮЧЕНИЯ ЧЕРЕЗ throw

- Это новый метод, который есть **у итераторов, созданных через генераторы**
- Он осуществляет **выполнение функции с места последнего yield**, как и next
- Но при запуске функции **в месте остановки происходит исключение**
- Объект исключения задаётся параметром throw
- Это **исключение можно перехватить** через try catch
- throw позволяет переводить наш КА в новое «качественное» состояние

```
function* example() {
  while (true) {
    try {
      for (let i = 0; i < 10; i++) {
        yield i;
      }

      return;

    } catch (reason) {
      switch (reason) {
        case "reset": break;
        default: throw reason;
      }
    }
  }
}

const iter = example();

console.log(iter.next());              // { value: 0, done: false }
console.log(iter.next());              // { value: 1, done: false }
console.log(iter.throw("reset")); // { value: 0, done: false }
console.log(iter.next());              // { value: 1, done: false }

try {
  console.log(iter.throw("boom!"));

} catch (err) {
  console.error(err); // boom!
}

// Итератор перешел в done: true
console.log(iter.next()); // { value: undefined, done: true }
console.log(iter.throw("boom!")); // Все еще выбрасывает исключение
```

РЕФЛЕКСИРУЕМ

- Благодаря методу throw мы можем легко организовать **любое дополнительное поведение генератора**
- Например, очистку или приостановку
- Главное помнить, что если исключение не обработает сама функция-генератор, то **оно должно быть обработано снаружи**
- А сам итератор после этого **переходит в завершённое состояние**

НЕ ТОЛЬКО yield

- Если оператор yield позволяет **приостановить выполнение функции** до следующего вызова методов итератора
- То оператор yield* выступает в роли улучшенного yield и может **делегировать работу другому Iterable объекту**
- Это очень **удобно для создания композиций**

```
function* example() {
    yield* [1, 2, 3];

    // То же самое, что и
    for (const el of [1, 2, 3]) {
        yield el;
    }
}

const iter = example();

console.log(iter.next()); // {done: false, value: 1}
console.log(iter.next()); // {done: false, value: 2}
console.log(iter.next()); // {done: false, value: 3}
```


second monitor
$0.00 (0%)
$0.00
224 DAYS TO 20
$160.00
СЛОЖНА, СЛОЖНАААА!

ВЫДЫХАЕМ

- Генераторы — сложная тема
- Сегодня наша задача — понять, **какие возможности имеют генераторы**
- А в других лекциях мы будем разбирать, что теперь мы можем с этим делать
- Пока держите в голове, что **генераторы — это «синтаксический сахар»** для создания итераторов

А КАК РАБОТАЕТ CAM yield?



А ВОТ ТУТ ИНТЕРЕСНО

- Оператор yield не просто возвращает значение итератора –
  он **приостанавливает выполнение функции** до следующего вызова метода next()
- С технической точки зрения **приостановить функцию невозможно**,
  так как функция имеет детерминированные состояния: вычислено или вычисляется
- Связано это в первую очередь с **организацией памяти функций на уровне стека** –
  мы просто не можем выполнить часть функции и потом вернуться к ней
- Если функция вернула значение, то её стековый кадр уничтожается

РЕШЕНИЕ ЧЕРЕЗ КОНЕЧНЫЙ АВТОМАТ

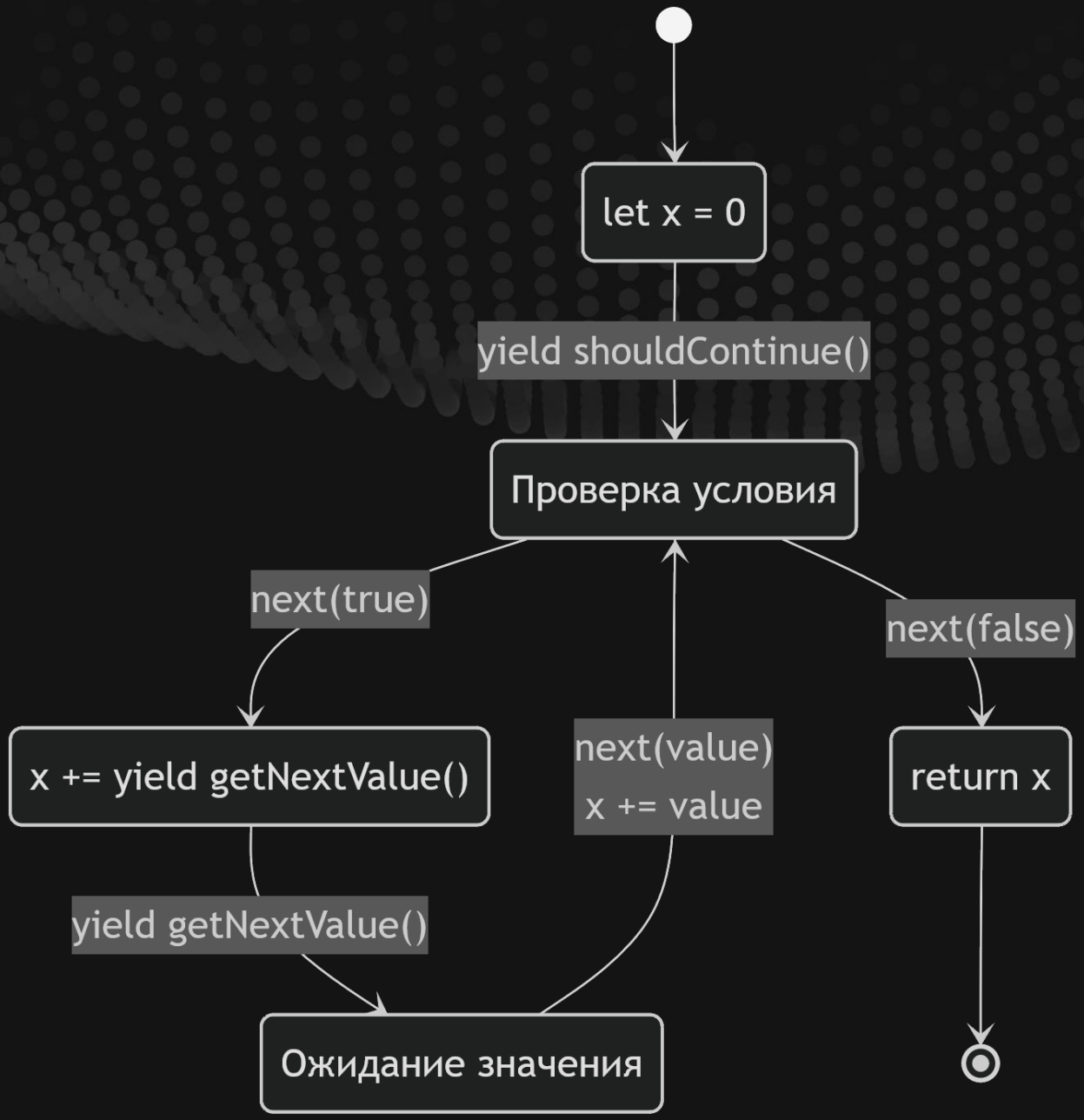
- **Разобьём код функции на «этапы»**, где разделителем выступает любой yield
- Представим нашу функцию в виде структуры, где **каждому этапу соответствует своя функция или ветка** switch case
- Функция **будет ожидать аргументом объект-состояния**, полученный от прошлого этапа
- В конце своей работы функция может **перевести вычисление на другой «этап»**

НАПРИМЕР

```
function* example(shouldContinue, getNextValue) {
  let x = 0;

  while (yield shouldContinue()) {
    x += yield getNextValue();
  }

  return x;
}
```


let x = 0
yield shouldContinue()
Проверка условия
next(true)
next(false)
x += yield getNextValue()
next(value)
x += value
return x
yield getNextValue()
Ожидание значения

```
var _marked = /*#__PURE__*/regeneratorRuntime.mark(example);
function example(shouldContinue, getNextValue) {
  var x;
  return regeneratorRuntime.wrap(function example$(_context) {
    while (1) {
      switch (_context.prev = _context.next) {
        case 0:
          x = 0;
        case 1:
          _context.next = 3;
          return shouldContinue();
        case 3:
          if (!_context.sent) {
            _context.next = 10;
            break;
          }
          _context.t0 = x;
          _context.next = 7;
          return getNextValue();
        case 7:
          x = _context.t0 += _context.sent;
          _context.next = 1;
          break;
        case 10:
          return _context.abrupt("return", x);
        case 11:
        case "end":
          return _context.stop();
      }
    }
  }, _marked);
}
```

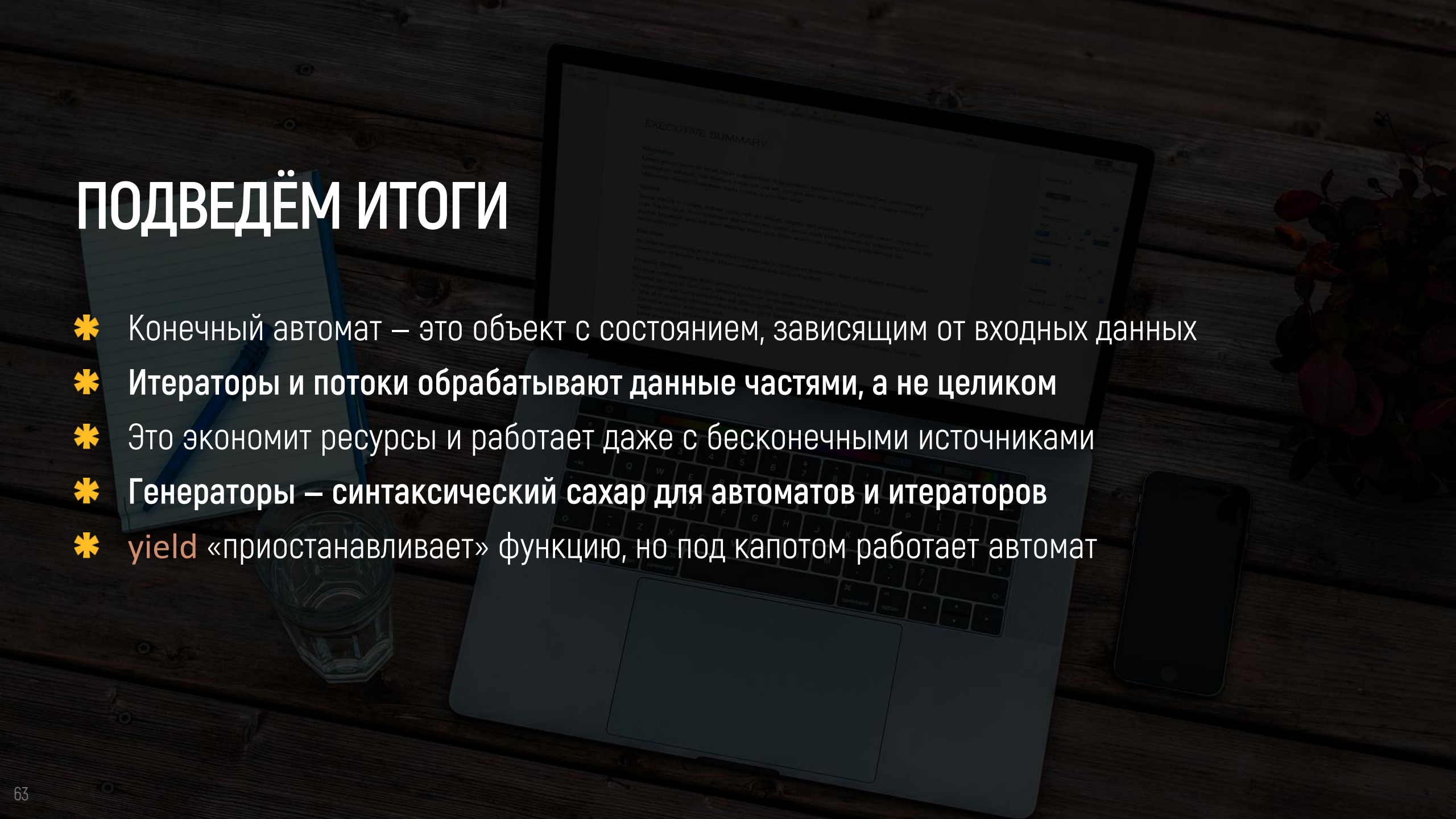

second monitor
$0.00 (0%)
$0.00
224 DAYS TO 20
$160.00
СЛОЖНА, СЛОЖНАААА!

ВЫДЫХАЕМ

- На прошлом примере мы видели реальный код полифила-трансформера для генераторов
- Но похожим образом оно работает и нативно
- Мы не можем «приостановить» функцию, поэтому **разбиваем вычисление на несколько вызовов КА**
- **Передаём состояние вычисления** от функции к функции

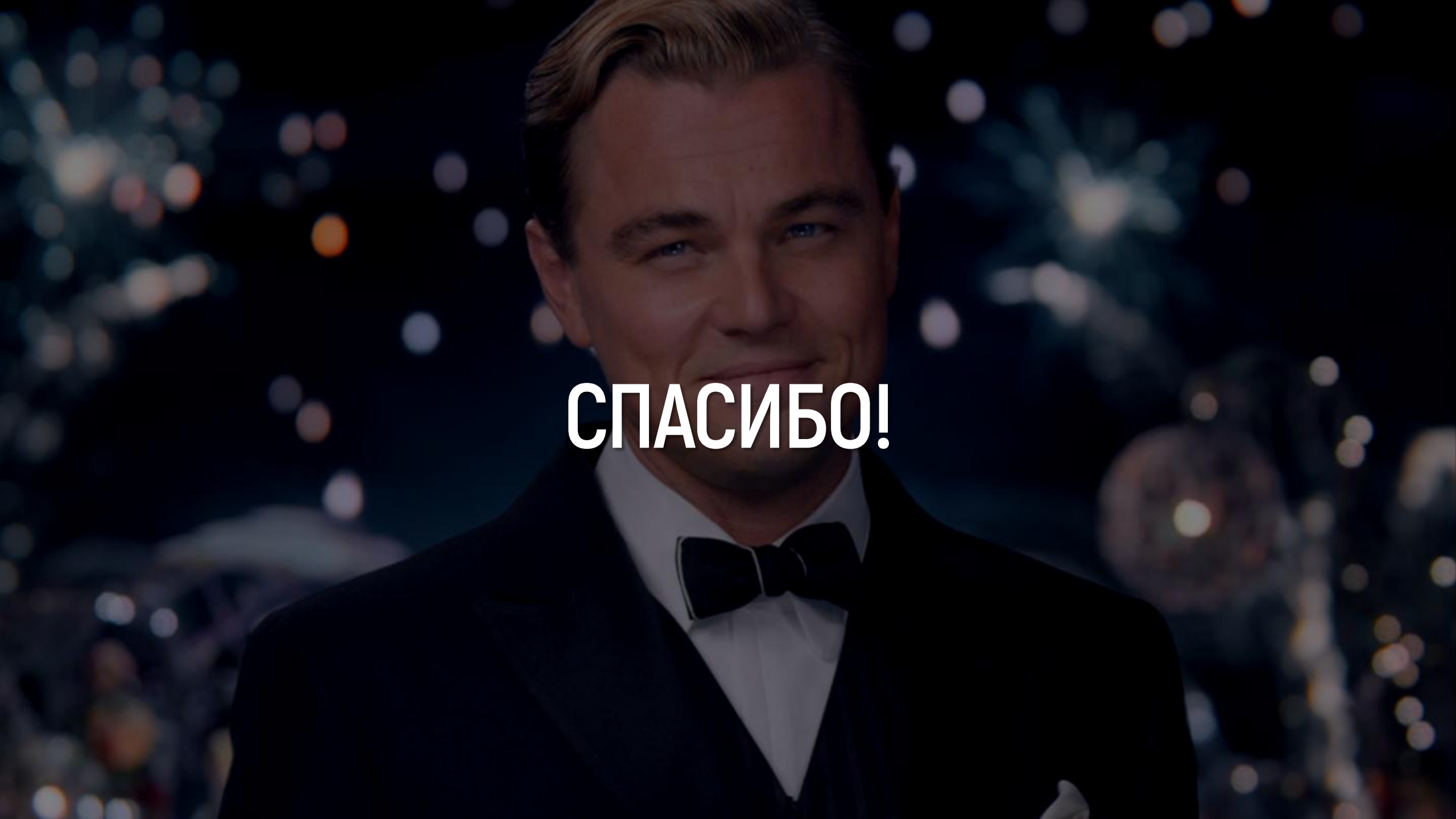
РЕФЛЕКСИРУЕМ

- Генераторы — это удобный **«синтаксический сахар» для описания автоматов**
- Под капотом **сам генератор реализует обычный автомат**
- **Состояние автомата сохраняется** как отдельная структура в куче
- Но концептуально мы научились **приостанавливать функции**
- У этого есть очень интересные сферы применения



ПОДВЕДЁМ ИТОГИ

- Конечный автомат — это объект с состоянием, зависящим от входных данных
- **Итераторы и потоки обрабатывают данные частями, а не целиком**
- Это экономит ресурсы и работает даже с бесконечными источниками
- **Генераторы — синтаксический сахар для автоматов и итераторов**
- yield «приостанавливает» функцию, но под капотом работает автомат


СПАСИБО!